BENCHMARKING POST-TRAINING QUANTIZATION DAN QUANTIZATION-AWARE TRAINING PADA ARSITEKTUR YOLO MODERN UNTUK INFERENSI EDGE HETEROGEN

SKRIPSI

Diajukan untuk memenuhi salah satu syarat memperoleh gelar
Sarjana Teknik pada Program Studi Teknik Komputer
Universitas Darussalam Gontor

Oleh:
Fatih Jawwad Al-Mumtaz
NIM: XXXXXXXXXX

PROGRAM STUDI TEKNIK KOMPUTER
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS DARUSSALAM GONTOR
PONOROGO
2026

HALAMAN PERSETUJUAN

Skripsi berjudul *"Benchmarking Post-Training Quantization dan Quantization-Aware Training pada Arsitektur YOLO Modern untuk Inferensi Edge Heterogen"* telah disetujui oleh pembimbing untuk diujikan.

Ponorogo, ... 2025

Dosen Pembimbing
NIP: . . . . . .

ABSTRAK

Fatih Jawwad Al-Mumtaz. *Benchmarking Post-Training Quantization dan Quantization-Aware Training pada Arsitektur YOLO Modern untuk Inferensi Edge Heterogen.* Skripsi. Program Studi Teknik Komputer, Universitas Darussalam Gontor, Ponorogo. 2026.

Deployment model deteksi objek berbasis YOLO pada perangkat edge dengan sumber daya terbatas memerlukan teknik optimasi inferensi yang tepat. Dua pendekatan utama yang umum digunakan adalah *Post-Training Quantization* (PTQ) dan *Quantization-Aware Training* (QAT), yang masing-masing memiliki trade-off antara akurasi, latensi, dan biaya komputasi. Penelitian ini melakukan benchmarking sistematis terhadap PTQ dan QAT pada arsitektur YOLOv8n dan YOLOv26n di tiga platform hardware heterogen: Apple Silicon M4 (Mac Mini M4), GPU NVIDIA RTX 4060, dan MCU-class ESP32-S3. Metrik yang diukur meliputi latensi (mean, p95, p99), throughput (FPS), akurasi (mAP@0.5), ukuran model, dan konsumsi daya. Hasil penelitian diharapkan dapat memberikan panduan praktis bagi pengembang sistem edge AI dalam memilih teknik optimasi yang sesuai dengan karakteristik hardware target.

Kata kunci: kuantisasi, YOLO, edge AI, inferensi heterogen, PTQ, QAT, Apple Silicon, ESP32

ABSTRACT

Fatih Jawwad Al-Mumtaz. *Benchmarking Post-Training Quantization and Quantization-Aware Training on Modern YOLO Architectures for Heterogeneous Edge Inference.* Undergraduate Thesis. Department of Computer Engineering, Universitas Darussalam Gontor, Ponorogo. 2026.

Deploying YOLO-based object detection models on resource-constrained edge devices requires careful selection of inference optimization techniques. Post-Training Quantization (PTQ) and Quantization-Aware Training (QAT) are the two dominant approaches, each presenting distinct trade-offs between accuracy, latency, and computational cost. This study conducts a systematic benchmark of PTQ and QAT on YOLOv8n and YOLOv26n across three heterogeneous hardware platforms: Apple Silicon M4 (Mac Mini M4), NVIDIA RTX 4060 GPU, and MCU-class ESP32-S3. Evaluation metrics include latency (mean, p95, p99), throughput (FPS), accuracy (mAP@0.5), model size, and power consumption. Findings are expected to provide actionable guidance for edge AI developers in selecting optimization strategies suited to their target hardware.

Keywords: quantization, YOLO, edge AI, heterogeneous inference, PTQ, QAT, Apple Silicon, ESP32

Daftar Isi

Halaman Persetujuan **ii**

Abstrak **iii**

Abstract **iv**

1 PENDAHULUAN **1**

1.1 Latar Belakang 1
1.2 Rumusan Masalah 2
1.3 Tujuan Penelitian 2
1.4 Batasan Penelitian 2
1.5 Manfaat Penelitian 3
1.6 Sistematika Penulisan 3

2 TINJAUAN PUSTAKA **4**

2.1 Deteksi Objek Real-Time 5
2.2 Kuantisasi Model Neural Network 5
2.2.1 Post-Training Quantization (PTQ) 5
2.2.2 Quantization-Aware Training (QAT) 5
2.2.3 Perbandingan PTQ dan QAT 5
2.3 Arsitektur YOLO 5
2.3.1 YOLOv8 5
2.3.2 YOLOv26n 5
2.4 Platform Edge Computing 5
2.4.1 Apple Silicon M4 5
2.4.2 NVIDIA RTX 4060 5
2.4.3 ESP32-S3 5
2.5 Framework Inferensi 5
2.6 Penelitian Terkait 5

3 METODOLOGI PENELITIAN **6**
3.1 Desain Penelitian 6
3.2 Dataset 6
3.2.1 Dataset Kalibrasi PTQ 6
3.2.2 Dataset Evaluasi Akurasi 6
3.3 Model 7
3.3.1 YOLOv8n 7
3.3.2 YOLOv26n 7
3.4 Pipeline Eksperimen 7
3.5 Prosedur Benchmark 7
3.5.1 Pengukuran Latensi 7
3.5.2 Pengukuran Akurasi 7
3.5.3 Pengukuran Konsumsi Daya 8
3.6 Experiment Tracking 8

4 HASIL DAN PEMBAHASAN **9**
4.1 Hasil Benchmark Mac Mini M4 9
4.2 Hasil Benchmark RTX 4060 10
4.3 Hasil Benchmark ESP32-S3 10
4.4 Analisis Komparatif Cross-Platform 10
4.5 Pembahasan 10

5 KESIMPULAN DAN SARAN **11**
5.1 Kesimpulan 11
5.2 Saran 11

Daftar Gambar

Daftar Tabel

2.1 Perbandingan Penelitian Terkait 5

4.1 Hasil Benchmark YOLOv8n pada Mac Mini M4 9
4.2 Hasil Benchmark YOLOv26n pada Mac Mini M4 9

Bab 1

PENDAHULUAN

1.1 Latar Belakang

Perkembangan model deteksi objek berbasis deep learning, khususnya keluarga YOLO (*You Only Look Once*), telah memungkinkan penerapan computer vision secara real-time pada berbagai aplikasi praktis seperti pengawasan, robotika, dan sistem otonom [1]. Namun, deployment model-model ini pada perangkat edge dengan sumber daya komputasi terbatas tetap menjadi tantangan signifikan, terutama karena tingginya kebutuhan memori dan throughput komputasi dari model berperforma tinggi [2].

Teknik kompresi model, terutama kuantisasi bobot dan aktivasi, telah terbukti efektif dalam mengurangi ukuran model dan meningkatkan kecepatan inferensi tanpa kehilangan akurasi yang signifikan [3]. Dua pendekatan utama yang banyak diteliti adalah *Post-Training Quantization* (PTQ) dan *Quantization-Aware Training* (QAT). PTQ mengkuantisasi model setelah pelatihan selesai menggunakan dataset kalibrasi kecil, sehingga tidak memerlukan proses pelatihan ulang [4]. Sebaliknya, QAT mengintegrasikan efek kuantisasi selama proses pelatihan sehingga model dapat beradaptasi terhadap error kuantisasi, menghasilkan akurasi yang lebih tinggi terutama pada presisi rendah [2].

Meskipun perbandingan PTQ dan QAT telah banyak diteliti, terdapat kesenjangan literatur yang signifikan pada kombinasi hardware tertentu. Studi yang ada umumnya berfokus pada platform x86 CPU atau GPU NVIDIA [5], smartphone berbasis ARM [6], atau Raspberry Pi [7]. Sejauh penelusuran yang dilakukan, belum terdapat studi yang secara sistematis membandingkan PTQ dan QAT pada arsitektur YOLO modern di platform Apple Silicon M4 dengan Neural

Engine, GPU NVIDIA RTX 4060, dan perangkat MCU-class ESP32-S3 secara bersamaan dalam satu kerangka eksperimen yang unified.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah penelitian ini adalah:

1. Bagaimana trade-off latensi, throughput, akurasi, dan konsumsi daya antara PTQ dan QAT pada arsitektur YOLOv8n dan YOLOv26n di platform Apple Silicon M4?
2. Bagaimana perbandingan karakteristik inferensi PTQ dan QAT antara platform Apple Silicon M4, GPU NVIDIA RTX 4060, dan MCU-class ESP32-S3?
3. Format kuantisasi mana (FP32, FP16, PTQ INT8, QAT INT8) yang memberikan keseimbangan optimal antara akurasi dan efisiensi inferensi pada masing-masing platform?

1.3 Tujuan Penelitian

1. Mengukur dan membandingkan latensi, throughput, akurasi, dan konsumsi daya PTQ dan QAT pada YOLOv8n dan YOLOv26n di tiga platform hardware heterogen.
2. Mengidentifikasi karakteristik hardware-specific yang mempengaruhi efektivitas masing-masing teknik kuantisasi.
3. Memberikan panduan berbasis data empiris untuk pemilihan teknik optimasi inferensi pada sistem edge AI.

1.4 Batasan Penelitian

1. Model dibatasi pada YOLOv8n dan YOLOv26n.
2. Platform hardware: Mac Mini M4, NVIDIA RTX 4060, dan ESP32-S3 Sense.
3. Framework inferensi: ONNX Runtime, OpenVINO, TensorFlow Lite, dan CoreML.
4. Dataset evaluasi: subset COCO val2017 dan dataset RescueVision.

5. Pruning dan knowledge distillation berada di luar cakupan penelitian ini.

1.5 Manfaat Penelitian

1. Referensi empiris bagi pengembang sistem edge AI dalam memilih teknik optimasi inferensi sesuai karakteristik hardware.
2. Benchmark baseline pada platform Apple Silicon M4 dan ESP32-S3 yang belum tersedia dalam literatur yang ada.
3. Pipeline benchmarking yang reproducible dan open-source sebagai kontribusi komunitas.

1.6 Sistematika Penulisan

Skripsi ini disusun dalam lima bab. Bab I memaparkan latar belakang, rumusan masalah, tujuan, batasan, dan manfaat penelitian. Bab II mengulas landasan teori dan kajian pustaka terkait kuantisasi model, arsitektur YOLO, dan platform edge computing. Bab III menjelaskan metodologi penelitian meliputi desain eksperimen, pipeline benchmark, dan prosedur pengukuran. Bab IV menyajikan hasil eksperimen dan analisis komparatif. Bab V memuat kesimpulan dan saran untuk penelitian selanjutnya.

Bab 2

TINJAUAN PUSTAKA

2.1 Deteksi Objek Real-Time

2.2 Kuantisasi Model Neural Network

2.2.1 Post-Training Quantization (PTQ)

2.2.2 Quantization-Aware Training (QAT)

2.2.3 Perbandingan PTQ dan QAT

2.3 Arsitektur YOLO

2.3.1 YOLOv8

2.3.2 YOLOv26n

2.4 Platform Edge Computing

2.4.1 Apple Silicon M4

2.4.2 NVIDIA RTX 4060

2.4.3 ESP32-S3

2.5 Framework Inferensi

2.6 Penelitian Terkait

Tabel 2.1: Perbandingan Penelitian Terkait

Penelitian	Model	Hardware	Teknik	Gap

Bab 3

METODOLOGI PENELITIAN

3.1 Desain Penelitian

Penelitian ini menggunakan pendekatan eksperimental kuantitatif dengan desain benchmarking sistematis. Variabel bebas adalah teknik kuantisasi (FP32, FP16, PTQ INT8, QAT INT8) dan platform hardware (M4, RTX 4060, ESP32-S3). Variabel terikat adalah latensi inferensi, throughput, akurasi deteksi, ukuran model, dan konsumsi daya.

3.2 Dataset

3.2.1 Dataset Kalibrasi PTQ

Dataset kalibrasi PTQ menggunakan 500 gambar yang diambil secara acak dari COCO train2017. Ukuran ini dipilih berdasarkan rekomendasi dari literatur yang menunjukkan bahwa 100–500 gambar cukup untuk kalibrasi PTQ yang stabil [4].

3.2.2 Dataset Evaluasi Akurasi

Evaluasi akurasi menggunakan dua dataset:

1. COCO val2017 (5.000 gambar) untuk perbandingan dengan literatur yang ada.
2. Dataset RescueVision (domain-specific rescue imagery) untuk evaluasi pada use case yang relevan.

3.3 Model

3.3.1 YOLOv8n

YOLOv8n digunakan sebagai baseline yang well-established dengan referensi literatur yang luas, memungkinkan perbandingan langsung dengan studi sebelumnya.

3.3.2 YOLOv26n

YOLOv26n merepresentasikan generasi terbaru arsitektur YOLO. Baseline model telah tersedia dari pipeline RescueVision dengan mAP@0.5 = 0.528 dan ukuran ONNX 11.70MB.

3.4 Pipeline Eksperimen

Pipeline eksperimen terdiri dari empat tahap utama:

1. **Export**: Model PyTorch diekspor ke format ONNX sebagai representasi intermediate.
2. **Optimasi**: Dari ONNX, model dioptimasi ke format target (FP16, PTQ INT8, QAT INT8) menggunakan framework yang sesuai per platform.
3. **Benchmark**: Inferensi dijalankan dengan 10 iterasi warmup diikuti 100 iterasi pengukuran per konfigurasi.
4. **Logging**: Semua hasil dicatat ke MLflow untuk reproducibility.

3.5 Prosedur Benchmark

3.5.1 Pengukuran Latensi

Latensi diukur menggunakan `time.perf_counter()` dengan input dummy berukuran $640 \times 640 \times 3$. Warmup sebanyak 10 iterasi dilakukan sebelum pengukuran untuk menghilangkan efek cold start. Statistik yang dicatat: mean, standar deviasi, p95, p99.

3.5.2 Pengukuran Akurasi

Akurasi diukur menggunakan mAP@0.5 dan mAP@0.5:0.95 pada dataset evaluasi dengan confidence threshold 0.25 dan IoU threshold 0.45.

3.5.3 Pengukuran Konsumsi Daya

Konsumsi daya diukur menggunakan:

- Mac Mini M4: `powermetrics` (macOS)
- RTX 4060: `nvidia-smi` power query
- ESP32-S3: USB inline power meter

3.6 Experiment Tracking

Seluruh eksperimen ditracking menggunakan MLflow dengan parameter logging meliputi: nama model, format kuantisasi, platform hardware, framework inferensi, dan seluruh metrik hasil pengukuran. Dataset dan model checkpoint di-version menggunakan DVC.

Bab 4

HASIL DAN PEMBAHASAN

4.1 Hasil Benchmark Mac Mini M4

Tabel 4.1: Hasil Benchmark YOLOv8n pada Mac Mini M4

Format	**Latency mean (ms)**	**p99 (ms)**	**FPS**	**mAP@0.5**
FP32	–	–	–	–
FP16	–	–	–	–
PTQ INT8	–	–	–	–
QAT INT8	–	–	–	–

Tabel 4.2: Hasil Benchmark YOLOv26n pada Mac Mini M4

Format	**Latency mean (ms)**	**p99 (ms)**	**FPS**	**mAP@0.5**
FP32	–	–	–	–
FP16	–	–	–	–
PTQ INT8	–	–	–	–
QAT INT8	–	–	–	–

4.2 Hasil Benchmark RTX 4060

4.3 Hasil Benchmark ESP32-S3

4.4 Analisis Komparatif Cross-Platform

4.5 Pembahasan

Bab 5

KESIMPULAN DAN SARAN

5.1 Kesimpulan

5.2 Saran

Bibliografi

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 779–788.

[2] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713.

[3] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding," in *International Conference on Learning Representations*, 2016.

[4] R. Banner, Y. Nahshan, and D. Soudry, "Post training 4-bit quantization of convolutional networks for rapid-deployment," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[5] M. Wang, H. Sun, J. Shi, X. Liu, B. Zhang, and X. Cao, "Q-YOLO: Efficient inference for real-time object detection," in *Asian Conference on Pattern Recognition*, 2023, pp. 307–321.

[6] G. Idama, Y. Guo, and W. Yu, "QATFP-YOLO: Optimizing object detection on non-gpu devices with yolo using quantization-aware training and filter pruning," in *2024 33rd International Conference on Computer Communications and Networks (ICCCN)*. IEEE, 2024.

[7] S. Boddu and A. Mukherjee, "Efficient edge deployment of quantized YOLOv4-Tiny for aerial emergency object detection on Raspberry Pi 5," in *IEEE Conference*, 2025.

[8] Y. Seo, J. Kim, J.-K. Kang, and Y. Kim, "LLTQ+: A hardware-friendly quantization framework for modern yolo architectures," *IEEE Access*, vol. 13, pp. 151 189–151 201, 2025.